RESPOSTAS — 50 EXERCÍCIOS

Arquitetura e processos de desenvolvimento • Linguagens de programação • SGBDs • Ferramentas CASE

Material elaborado com respostas explicativas e exemplos práticos, seguindo a ordem e os temas do arquivo enviado.

Parte I — Arquitetura de sistemas (1–10)

01. Defina arquitetura de software e explique sua importância para manutenção, desempenho e evolução de um sistema.

Arquitetura de software é a organização fundamental de um sistema: seus componentes, responsabilidades, relações e formas de comunicação. Ela funciona como uma estrutura orientadora para a construção da aplicação. Uma boa arquitetura facilita a manutenção porque separa responsabilidades e torna alterações mais localizadas; contribui para o desempenho ao permitir escolher estruturas adequadas ao volume de uso; e favorece a evolução porque possibilita adicionar funcionalidades ou substituir componentes sem precisar reconstruir todo o sistema.

02. Diferencie arquitetura monolítica, cliente-servidor e microsserviços. Apresente uma vantagem e uma limitação de cada abordagem.

Monolítica: a aplicação é desenvolvida e normalmente implantada como uma única unidade. Vantagem: simplicidade inicial de desenvolvimento e implantação. Limitação: com o crescimento, alterações e escalabilidade podem ficar mais difíceis, pois partes diferentes compartilham a mesma unidade.
Cliente-servidor: clientes solicitam serviços e um servidor processa as requisições e fornece dados ou funcionalidades. Vantagem: centralização de regras e dados, com vários clientes podendo acessar o mesmo serviço. Limitação: o servidor ou a rede podem se tornar gargalos ou pontos de indisponibilidade.
Microsserviços: o sistema é dividido em serviços menores, organizados por capacidades de negócio e comunicados por APIs ou mensagens. Vantagem: serviços podem ser implantados e escalados de forma independente. Limitação: aumenta a complexidade operacional, de comunicação, monitoramento e consistência dos dados.

03. Explique as responsabilidades das camadas de apresentação, negócio e persistência em uma arquitetura de três camadas.

Apresentação: interage com o usuário, exibe informações e recebe entradas, como uma tela web ou aplicativo. **Negócio:** aplica regras e validações, por exemplo verificar se um empréstimo pode ser realizado. **Persistência:** armazena e recupera dados em banco de dados ou outro meio permanente. Em geral, a apresentação solicita uma operação à camada de negócio, que consulta ou altera dados por meio da persistência, e o resultado retorna à interface.

04. Compare escalabilidade vertical e horizontal e indique um cenário adequado para cada uma.

Escalabilidade vertical consiste em aumentar os recursos de uma máquina, como CPU, memória e armazenamento. É adequada, por exemplo, para um sistema pequeno ou médio cujo banco de dados pode ser atendido por um servidor mais potente. Escalabilidade horizontal consiste em adicionar mais máquinas ou instâncias para dividir a carga. É adequada para um site de comércio eletrônico que recebe muitos acessos simultâneos e pode distribuir requisições entre vários servidores. A vertical tem limite físico e pode exigir máquina maior; a horizontal exige mecanismos de distribuição e coordenação.

05. Explique os conceitos de acoplamento e coesão. Por que baixo acoplamento e alta coesão são desejáveis?

Acoplamento é o grau de dependência entre módulos. Coesão é o quanto as responsabilidades de um módulo estão relacionadas entre si. Baixo acoplamento significa que uma alteração em um módulo tende a afetar menos os demais. Alta coesão significa que cada módulo tem um propósito claro e reúne funções relacionadas. Essas características facilitam testes, manutenção, reutilização e compreensão do código.

06. Desenhe a arquitetura de três camadas de um sistema de biblioteca: interface, regras de empréstimo e banco de dados. Identifique as trocas de informação.

A interface recebe o pedido de empréstimo e apresenta o resultado. A camada de negócio verifica disponibilidade, situação do leitor e limite de empréstimos. A persistência consulta livros e leitores e grava o empréstimo e a data prevista de devolução. O fluxo de retorno leva a confirmação ou a mensagem de erro até a interface.

```
[Usuário]
    |
    v
[Apresentação: tela da biblioteca]
    | solicita empréstimo (leitor_id, livro_id)
    v
[Negócio: regras de empréstimo]
    |-- valida leitor e disponibilidade
    |-- calcula prazo de devolução
    | consulta/grava dados
    v
[Persistência: banco de dados]
    | retorna dados / confirmação
    v
[Negócio] -> resultado -> [Apresentação] -> [Usuário]
```

07. Para um aplicativo de delivery, identifique ao menos cinco componentes e descreva suas responsabilidades.

1. Aplicativo do cliente: permite consultar restaurantes, montar carrinho e acompanhar pedidos. 2. API/gateway: recebe requisições, encaminha chamadas aos serviços e aplica controles básicos. 3. Autenticação: valida identidade e permissões dos usuários. 4. Serviço de restaurantes e catálogo: mantém estabelecimentos, cardápios e preços. 5. Serviço de pedidos: registra pedidos e controla seus estados. 6. Serviço de pagamentos: integra meios de pagamento e registra confirmações. 7. Serviço de entregas: associa entregadores, acompanha coleta e entrega. 8. Banco de dados: persiste usuários, produtos, pedidos e transações. Os componentes podem ser módulos de um monólito ou serviços separados, conforme a arquitetura escolhida.

08. Um sistema de matrícula apresenta lentidão em períodos de pico. Proponha duas mudanças arquiteturais e justifique como melhorariam a capacidade de atendimento.

Primeira mudança: colocar múltiplas instâncias da aplicação atrás de um balanceador de carga. As requisições são distribuídas entre servidores, evitando concentrar todo o tráfego em uma única instância. Segunda mudança: otimizar o acesso ao banco de dados, com índices nas colunas mais consultadas, consultas revisadas e cache para informações de leitura frequente. Isso reduz o tempo de resposta e a quantidade de consultas repetidas. As mudanças devem ser acompanhadas por testes de carga e monitoramento para identificar novos gargalos.

09. Desenhe um fluxo de requisição HTTP, desde o navegador até o banco de dados, para consultar o histórico escolar de um aluno.

O navegador envia uma requisição HTTPS à aplicação. O servidor autentica o usuário e verifica sua permissão para consultar o histórico. A camada de negócio solicita as matrículas e notas à camada de persistência, que executa uma consulta no banco. Os dados retornam ao servidor, que monta uma resposta HTTP (por exemplo, JSON ou uma página HTML) e a envia ao navegador.

```
Navegador
   | GET /alunos/123/historico (HTTPS)
   v
Servidor web / API
   | autentica e autoriza
   v
Serviço de Histórico (regras de negócio)
   | consulta dados
   v
Camada de persistência
   | SELECT de disciplinas, matrículas e notas
   v
Banco de dados
   | linhas encontradas
   v
Persistência -> Serviço -> API
   | resposta HTTP 200 (JSON/HTML)
   v
Navegador exibe o histórico
```

10. Crie uma tabela comparando monólito e microsserviços segundo implantação, testes, comunicação, custos operacionais e tolerância a falhas.

Critério	Monólito	Microsserviços
Implantação	Normalmente uma unidade; publicação simples.	Serviços podem ser publicados separadamente; exige orquestração.
Testes	Testes integrados podem ser mais simples no início.	Além dos testes de cada serviço, exige testes de integração e contrato.
Comunicação	Chamadas internas, geralmente no mesmo processo.	Comunicação por rede, APIs ou mensageria.
Custos operacionais	Em geral menores para projetos pequenos.	Podem ser maiores por infraestrutura, observabilidade e operação.
Tolerância a falhas	Falha grave pode afetar toda a aplicação.	Falhas podem ficar isoladas, se houver resiliência bem projetada.

Parte II — Processos de desenvolvimento (11–20)

11. Descreva as etapas de levantamento de requisitos, análise, projeto, implementação, testes, implantação e manutenção.

Levantamento de requisitos: descobrir necessidades, objetivos e restrições dos usuários. Análise: organizar, detalhar e verificar a viabilidade e a consistência dos requisitos. Projeto: definir arquitetura, componentes, interfaces, banco de dados e tecnologias. Implementação: transformar o projeto em código. Testes: verificar funcionalidades, integração, segurança e qualidade, identificando defeitos. Implantação: disponibilizar o sistema no ambiente de uso e preparar usuários e infraestrutura. Manutenção: corrigir problemas, adaptar o sistema a mudanças e implementar melhorias.

12. Compare os modelos cascata, incremental e espiral em relação a mudanças de requisitos e gestão de riscos.

Cascata: fases predominantemente sequenciais; funciona melhor quando os requisitos são estáveis, mas mudanças tardias costumam ser caras. A gestão de riscos tende a ocorrer por planejamento e revisões ao longo das fases. Incremental: o produto é entregue em partes funcionais; permite incorporar feedback e mudanças entre incrementos, reduzindo o risco de esperar até o final para validar o sistema. Espiral: desenvolvimento em ciclos que incluem definição de objetivos, análise de riscos, desenvolvimento e avaliação; é especialmente útil em projetos grandes ou de alto risco, embora tenha maior custo de gestão.

13. Explique a diferença entre requisito funcional e não funcional. Dê três exemplos de cada para um sistema hospitalar.

Requisito funcional descreve o que o sistema deve fazer. Exemplos: (1) cadastrar pacientes; (2) permitir agendamento e cancelamento de consultas; (3) registrar prescrições médicas. Requisito não funcional descreve qualidades, restrições ou condições de operação. Exemplos: (1) telas principais devem responder em até dois segundos sob carga definida; (2) acesso a prontuários deve exigir autenticação e autorização; (3) o serviço deve manter disponibilidade mensal de 99,9%, conforme condições acordadas.

14. Diferencie Scrum, Kanban e práticas de DevOps, indicando o foco principal de cada abordagem.

Scrum organiza o trabalho em ciclos curtos chamados sprints, com papéis, eventos e artefatos definidos; seu foco é entregar incrementos de produto e inspecionar/adaptar o trabalho. Kanban visualiza o fluxo em um quadro, limita o trabalho em andamento e busca melhorar a continuidade e o tempo de entrega; seu foco é otimizar o fluxo. DevOps é um conjunto de práticas culturais e técnicas que aproxima desenvolvimento e operações, com automação, integração e entrega contínuas, monitoramento e colaboração; seu foco é entregar software com frequência e confiabilidade.

15. Explique a diferença entre testes unitários, de integração, de sistema e de aceitação.

Testes unitários verificam pequenas unidades isoladas, como uma função de cálculo. Testes de integração verificam se módulos ou serviços funcionam corretamente em conjunto, por exemplo aplicação e banco de dados. Testes de sistema avaliam o sistema completo em relação aos requisitos funcionais e não funcionais. Testes de aceitação verificam, com critérios definidos pelo negócio ou usuário, se a solução atende à necessidade e está pronta para uso.

16. Escreva cinco histórias de usuário para um sistema acadêmico.

As histórias podem ser escritas da seguinte forma:

```
1. Como aluno, quero consultar minhas notas, para acompanhar meu desempenho.
2. Como aluno, quero visualizar minha grade de horários, para organizar minha rotina.
3. Como professor, quero lançar notas e faltas, para manter os registros das turmas atualizados.
4. Como secretário acadêmico, quero realizar matrículas, para registrar a participação dos alunos nas disciplinas.
5. Como coordenador, quero emitir relatórios de aprovação, para acompanhar os resultados dos cursos.
```

17. Monte um backlog com oito tarefas para desenvolver um cadastro de produtos. Classifique cada tarefa por prioridade e estimativa de esforço.

Tarefa	Prioridade	Esforço estimado
Definir campos e regras do produto	Alta	2 pontos
Criar tabela Produto no banco	Alta	3 pontos
Implementar inclusão de produto	Alta	5 pontos
Implementar consulta/listagem	Alta	3 pontos
Implementar edição de produto	Média	5 pontos
Implementar exclusão lógica	Média	3 pontos
Validar campos obrigatórios e preço	Alta	3 pontos
Criar testes e revisar a tela	Média	5 pontos

A estimativa em pontos é relativa: 2 representa tarefa pequena, 3 média e 5 uma tarefa que exige mais implementação ou validação.

18. Planeje uma sprint de duas semanas para uma tela de login.

Objetivo: entregar uma tela de login funcional, com validação de campos e autenticação integrada ao serviço disponível. Entregas: layout responsivo; campos de e-mail/usuário e senha; validação de preenchimento; integração com autenticação; mensagens de erro; testes. Critérios de aceite: usuário válido consegue entrar; senha incorreta não permite acesso; campos vazios são sinalizados; erros de conexão são informados sem expor dados internos; senha não é armazenada em texto puro. Riscos: atraso na disponibilização da API, regras de autenticação indefinidas e falhas de segurança. Mitigações: alinhar contrato da API no início, usar dados de teste e revisar armazenamento/transmissão de credenciais.

19. Escreva cinco casos de teste para a função recuperar senha, incluindo pré-condições, entradas e resultados esperados.

Pré-condição geral: usuário está na tela de recuperação e o serviço de envio de e-mail está configurado.

```
CT01 — E-mail cadastrado: entrada: usuario@exemplo.com cadastrado. Esperado: sistema confirma a solicitação e envia instr
CT02 — E-mail não cadastrado: entrada: inexistente@exemplo.com. Esperado: sistema apresenta mensagem neutra, sem revelar
CT03 — Campo vazio: entrada: nenhum e-mail. Esperado: validação informa que o campo é obrigatório e não envia solicitação
CT04 — Formato inválido: entrada: usuario@. Esperado: sistema solicita um endereço válido e não inicia a recuperação.
CT05 — Link expirado: pré-condição: link gerado anteriormente está fora da validade. Entrada: usuário acessa o link expir
```

20. Crie um quadro Kanban com as colunas solicitadas e distribua dez tarefas de um projeto fictício.

Projeto fictício: portal acadêmico. O quadro abaixo apresenta uma distribuição possível do trabalho no momento do acompanhamento.

```
A FAZER
- Criar relatório de notas
- Implementar recuperação de senha
- Documentar API

EM ANDAMENTO
- Desenvolver tela de matrícula
- Criar endpoint de consulta de aluno
- Ajustar layout para celular

EM TESTE
- Validar login
- Testar cadastro de disciplinas

CONCLUÍDO
- Criar estrutura inicial do banco
- Configurar repositório Git
```

Parte III — Linguagens de programação (21–30)

21. Diferencie linguagens compiladas e interpretadas, observando que muitas plataformas combinam compilação e execução por máquina virtual.

Em uma linguagem compilada, o código-fonte é traduzido previamente para uma forma executável, como código de máquina ou código intermediário. Em uma linguagem interpretada, um programa executor analisa e executa as instruções durante a execução. Na prática, a distinção não é absoluta: Java, por exemplo, compila o código para bytecode e a JVM o executa, podendo usar compilação JIT; Python normalmente transforma o código em bytecode e o executa em uma máquina virtual. Portanto, muitas plataformas combinam etapas de compilação e interpretação/execução.

22. Explique programação procedural, orientada a objetos e funcional, citando uma característica marcante de cada paradigma.

Procedural: organiza o programa em procedimentos ou funções que executam passos sobre dados; exemplo: uma sequência de funções para cadastrar e listar produtos. Orientada a objetos: organiza o sistema em classes e objetos que reúnem estado e comportamento; característica marcante: encapsulamento. Funcional: enfatiza funções, composição e transformação de valores, buscando reduzir efeitos colaterais; característica marcante: uso de funções puras e imutabilidade quando possível.

23. Compare tipagem estática e dinâmica, com exemplos de linguagens que utilizam cada abordagem.

Na tipagem estática, os tipos são verificados principalmente antes da execução, durante a compilação ou análise do programa; exemplos: C#, Java e C++. Na tipagem dinâmica, o tipo é associado ao valor em tempo de execução e uma variável pode receber valores de tipos diferentes; exemplos: Python e JavaScript. Tipagem estática pode detectar certos erros mais cedo; tipagem dinâmica pode oferecer flexibilidade, mas exige atenção aos valores usados em cada operação.

24. Descreva os papéis de HTML, CSS, JavaScript e de uma linguagem de back-end em uma aplicação web.

HTML define a estrutura e o conteúdo da página, como títulos, formulários e botões. CSS define a apresentação visual, incluindo cores, espaçamentos e adaptação a telas. JavaScript implementa interatividade no navegador, como validação de formulários e atualização de partes da página. Uma linguagem de back-end, como Python, Java, C# ou JavaScript no servidor, executa regras de negócio, autentica usuários e acessa serviços e bancos de dados.

25. Explique a diferença entre variável, constante, função, classe e objeto.

Variável é um identificador associado a um valor que pode mudar durante a execução. Constante representa um valor que não deve ser reatribuído após sua definição, conforme as regras da linguagem. Função é um bloco reutilizável que realiza uma tarefa e pode receber parâmetros e retornar um resultado. Classe é uma definição que descreve atributos e comportamentos de um tipo de objeto. Objeto é uma instância concreta de uma classe, com seus próprios valores de estado.

26. Escreva em Python um programa que leia três notas, calcule a média e informe se o aluno foi aprovado (média igual ou superior a 7).

O programa lê três valores, calcula a média aritmética e compara o resultado com 7. O exemplo considera notas numéricas válidas.

```
nota1 = float(input("Digite a primeira nota: "))
nota2 = float(input("Digite a segunda nota: "))
nota3 = float(input("Digite a terceira nota: "))

media = (nota1 + nota2 + nota3) / 3

print(f"Média: {media:.2f}")

if media >= 7:
    print("Aprovado")
else:
    print("Reprovado")
```

27. Crie uma função em Python que receba uma lista de preços e retorne o total, aplicando desconto de 10% quando o valor ultrapassar R$ 500.

A função soma os preços. Se a soma for maior que R$ 500, aplica 10% de desconto sobre o total; caso contrário, retorna o valor integral.

```
def calcular_total(precos):
    total = sum(precos)

    if total > 500:
        total = total * 0.90

    return total

# Exemplo de uso
valores = [200.00, 180.00, 150.00]
print(f"Total: R$ {calcular_total(valores):.2f}")
```

28. Implemente em JavaScript uma função que valide um formulário de cadastro, exigindo nome não vazio, e-mail com formato básico válido e senha de pelo menos oito caracteres.

A função recebe os campos, remove espaços extras do nome, verifica um formato básico de e-mail e exige senha com no mínimo oito caracteres. A validação no cliente melhora a experiência, mas o servidor também deve validar os dados.

```
function validarCadastro(nome, email, senha) {
  const nomeValido = nome.trim().length > 0;
  const emailValido = /^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(email);
  const senhaValida = senha.length >= 8;

  return {
    valido: nomeValido && emailValido && senhaValida,
    nomeValido,
    emailValido,
    senhaValida
  };
}

// Exemplo
const resultado = validarCadastro(
  "Ana Silva",
  "ana@email.com",
  "senha123"
);
console.log(resultado);
```

29. Escreva pseudocódigo para buscar um produto pelo código em uma lista. Informe o resultado quando encontrado e quando inexistente.

Uma busca linear percorre a lista até encontrar um produto com o código informado. Se terminar a lista sem encontrar correspondência, informa que o produto não existe.

```
INÍCIO
    leia codigoProcurado
    encontrado <- FALSO

    PARA CADA produto NA listaProdutos FAÇA
        SE produto.codigo = codigoProcurado ENTÃO
            escreva "Produto encontrado: ", produto.nome
            encontrado <- VERDADEIRO
            PARE
        FIM SE
    FIM PARA

    SE encontrado = FALSO ENTÃO
        escreva "Produto não encontrado"
    FIM SE
FIM
```

30. Modele uma classe Produto com código, nome, preço e método para calcular preço com desconto. Mostre um exemplo de uso.

Exemplo em Python. O método recebe o percentual de desconto e retorna o preço calculado, sem alterar o preço original armazenado no objeto.

```
class Produto:
    def __init__(self, codigo, nome, preco):
        self.codigo = codigo
        self.nome = nome
```

```
        self.preco = preco

    def calcular_preco_com_desconto(self, percentual):
        desconto = self.preco * (percentual / 100)
        return self.preco - desconto


produto = Produto(101, "Teclado", 150.00)
preco_final = produto.calcular_preco_com_desconto(10)

print(produto.nome)
print(f"Preço com desconto: R$ {preco_final:.2f}")
```

Parte IV — SGBDs: MySQL, Oracle e dBase (31–40)

31. Defina SGBD e explique os conceitos de tabela, registro, campo, chave primária e chave estrangeira.

SGBD é o software que permite definir, armazenar, consultar, atualizar e controlar o acesso a um banco de dados. Tabela organiza dados de um mesmo assunto em linhas e colunas. Registro é uma linha da tabela, representando uma ocorrência, como um aluno. Campo é uma coluna, como nome ou data de nascimento. Chave primária identifica unicamente cada registro. Chave estrangeira é um campo ou conjunto de campos que referencia uma chave de outra tabela, estabelecendo relacionamento e ajudando a manter a integridade referencial.

32. Compare MySQL, Oracle Database e dBase quanto a contexto histórico, aplicações típicas, modelo de dados e recursos disponíveis.

MySQL surgiu como um SGBD relacional, amplamente adotado em aplicações web e sistemas que utilizam SQL. Oferece tabelas relacionais, índices, transações com mecanismos como InnoDB, integridade referencial e recursos de replicação e administração.
Oracle Database é um SGBD relacional/objeto-relacional corporativo, usado em sistemas empresariais de grande porte, com recursos avançados de segurança, alta disponibilidade, transações, particionamento, administração e escalabilidade; sua oferta e licenciamento são diferentes dos de soluções comunitárias.
dBase é uma família histórica de sistemas de banco de dados associada ao modelo xBase e ao uso de arquivos/tabelas DBF, muito utilizada em aplicações desktop e ambientes de microcomputadores. É mais limitada em recursos modernos de servidor, concorrência e administração quando comparada aos SGBDs relacionais atuais. Portanto, não são equivalentes: diferem em época, arquitetura, escala de uso e conjunto de funcionalidades.

33. Explique integridade referencial e a importância das restrições NOT NULL, UNIQUE e FOREIGN KEY.

Integridade referencial garante que relacionamentos entre tabelas permaneçam consistentes: uma chave estrangeira deve apontar para um registro existente, ou ser nula quando isso for permitido. NOT NULL impede que uma coluna receba valor nulo, exigindo preenchimento. UNIQUE impede valores duplicados na coluna ou combinação de colunas definida como única, como e-mail. FOREIGN KEY declara a referência entre tabelas e permite ao SGBD controlar a validade dessa relação.

34. Descreva as propriedades ACID de transações e dê um exemplo de sua importância em operações financeiras.

Atomicidade: a transação ocorre por completo ou é desfeita. Consistência: a transação preserva as regras e restrições do banco. Isolamento: transações simultâneas não devem produzir interferências indevidas. Durabilidade: após confirmação, os dados permanecem gravados mesmo diante de falhas, conforme as garantias do SGBD. Exemplo: transferência de R$ 100 entre contas deve debitar uma conta e creditar a outra na mesma transação. Se o crédito falhar, o débito deve ser revertido, evitando perda parcial do valor.

35. Explique normalização e diferencie, em linhas gerais, primeira, segunda e terceira formas normais.

Normalização é o processo de organizar tabelas e relacionamentos para reduzir repetição desnecessária e evitar anomalias de inserção, atualização e exclusão. Primeira Forma Normal (1FN): cada campo contém um valor atômico e não há grupos repetidos em uma mesma linha. Segunda Forma Normal (2FN): além de estar na 1FN, cada atributo não-chave depende da chave primária inteira, e não apenas de parte de uma chave composta. Terceira Forma Normal (3FN): além de estar na 2FN, atributos não-chave não devem depender transitivamente da chave; em termos simples, não devem depender de outro atributo não-chave.

36. Modele um banco de dados de biblioteca com as entidades Livro, Leitor e Emprestimo. Defina atributos, chaves e relacionamentos.

Um leitor pode realizar vários empréstimos ao longo do tempo, e um livro pode aparecer em vários empréstimos em datas diferentes. Cada empréstimo relaciona um leitor a um livro. Para simplificar, o modelo considera um exemplar identificado por livro_id; em uma biblioteca com múltiplos exemplares, seria melhor separar Livro e Exemplar.

```
LIVRO
- livro_id (PK)
- titulo
- autor
- isbn (UNIQUE)

LEITOR
- leitor_id (PK)
- nome
- email (UNIQUE)
- telefone

EMPRESTIMO
- emprestimo_id (PK)
- livro_id (FK -> LIVRO.livro_id)
- leitor_id (FK -> LEITOR.leitor_id)
- data_emprestimo
- data_prevista_devolucao
- data_devolucao (pode ser NULL)

Relacionamentos:
LEITOR 1 -------- N EMPRESTIMO
LIVRO  1 -------- N EMPRESTIMO
```

37. Escreva comandos SQL compatíveis com MySQL para criar a tabela Aluno, definindo chave primária e unicidade do e-mail.

O comando abaixo cria um identificador auto incremental como chave primária e impede e-mails duplicados. O tipo DATE armazena a data de nascimento.

```
CREATE TABLE Aluno (
    id INT AUTO_INCREMENT PRIMARY KEY,
    nome VARCHAR(150) NOT NULL,
    email VARCHAR(255) NOT NULL UNIQUE,
    data_nascimento DATE
) ENGINE=InnoDB;
```

38. Escreva comandos INSERT para cadastrar três alunos e um SELECT que liste os nomes em ordem alfabética.

Os comandos inserem três registros e, em seguida, consultam os nomes em ordem crescente (A–Z).

```
INSERT INTO Aluno (nome, email, data_nascimento)
VALUES
  ('Ana Souza', 'ana@email.com', '2004-03-15'),
  ('Bruno Lima', 'bruno@email.com', '2003-08-22'),
  ('Carla Mendes', 'carla@email.com', '2005-01-10');

SELECT nome
FROM Aluno
ORDER BY nome ASC;
```

39. Considere tabelas Cliente(id, nome) e Pedido(id, cliente_id, valor). Escreva um SELECT com JOIN que mostre nome do cliente e valor do pedido.

O INNER JOIN relaciona Pedido.cliente_id com Cliente.id e retorna os pedidos que possuem um cliente correspondente.

```
SELECT
```

```
    Cliente.nome,
    Pedido.valor
FROM Cliente
INNER JOIN Pedido
    ON Pedido.cliente_id = Cliente.id;
```

40. Uma aplicação permite que duas pessoas atualizem o mesmo estoque. Descreva o problema de concorrência e proponha uma estratégia transacional para evitar inconsistências.

O problema ocorre quando duas transações leem o mesmo saldo antes de qualquer uma gravar a alteração. Por exemplo, ambas leem estoque 10 e cada uma vende 7; se as duas gravarem o resultado 3, o sistema registra 3 unidades, embora 14 tenham sido vendidas. Uma solução é executar a atualização em transação com bloqueio da linha (SELECT ... FOR UPDATE), validar a quantidade disponível e atualizar o estoque antes do COMMIT. Outra abordagem é uma atualização atômica condicionada, por exemplo UPDATE Produto SET estoque = estoque - 7 WHERE id = 1 AND estoque >= 7, verificando se uma linha foi afetada. Deve-se tratar rollback e conflitos.

Parte V — Ferramentas CASE e projeto integrador (41–50)

41. Defina ferramentas CASE e diferencie apoio às fases de análise, projeto, implementação e manutenção.

CASE (Computer-Aided Software Engineering) é o conjunto de ferramentas que apoia atividades de engenharia de software. Na análise, auxilia a registrar requisitos e processos. No projeto, permite criar diagramas UML, modelos de dados e arquitetura. Na implementação, algumas ferramentas geram código, integram modelos ao ambiente de desenvolvimento ou verificam padrões. Na manutenção, ajudam a consultar documentação, rastrear dependências, analisar alterações e compreender sistemas existentes.

42. Explique a utilidade de diagramas de casos de uso, classes e sequência na modelagem de sistemas.

Diagrama de casos de uso apresenta os atores externos e as funcionalidades que o sistema oferece, mostrando o escopo e as interações esperadas. Diagrama de classes representa a estrutura estática: classes, atributos, operações e relacionamentos. Diagrama de sequência mostra a troca de mensagens entre participantes ao longo do tempo para realizar um cenário, como autenticar um usuário.

43. Diferencie ferramentas de modelagem, geradores de código e ferramentas de engenharia reversa.

Ferramentas de modelagem permitem criar e manter representações do sistema, como diagramas UML e modelos entidade-relacionamento. Geradores de código transformam modelos ou definições em código-fonte ou estruturas iniciais, reduzindo trabalho repetitivo. Ferramentas de engenharia reversa analisam código, banco de dados ou executáveis para recuperar estruturas e produzir modelos/documentação de um sistema já existente. Algumas ferramentas podem reunir mais de uma dessas capacidades.

44. Apresente três benefícios e duas limitações do uso de ferramentas CASE em equipes de desenvolvimento.

Benefícios: (1) melhor comunicação visual entre equipe e clientes por meio de modelos padronizados; (2) documentação mais organizada e rastreável; (3) automação de tarefas repetitivas, como geração de código ou validação de modelos. Limitações: (1) custo de licença, treinamento e implantação, dependendo da ferramenta; (2) risco de modelos ficarem desatualizados ou de a equipe depender excessivamente da ferramenta. CASE apoia o trabalho, mas não substitui análise, decisões técnicas nem revisão humana.

45. Explique como rastreabilidade de requisitos e documentação técnica podem reduzir retrabalho.

Rastreabilidade liga cada requisito às partes do projeto que o implementam, aos testes que o verificam e às mudanças relacionadas. Assim, quando um requisito muda, a equipe identifica quais telas, serviços, tabelas e testes podem ser afetados. A documentação técnica registra decisões, arquitetura, interfaces e regras importantes. Juntas, essas práticas reduzem interpretações diferentes, alterações incompletas, duplicação de esforço e falhas na validação.

46. Crie um diagrama de casos de uso para um sistema de biblioteca com três atores e seis casos de uso.

Atores: Leitor, Bibliotecário e Administrador. Casos de uso: consultar catálogo, reservar livro, solicitar empréstimo, registrar devolução, cadastrar livro e gerenciar usuários. O diagrama textual abaixo representa as associações principais; pode ser reproduzido visualmente em diagrams.net ou Visual Paradigm.

```
                SISTEMA DE BIBLIOTECA
    Leitor ---------------- (Consultar catálogo)
    Leitor ---------------- (Reservar livro)
    Leitor ---------------- (Solicitar empréstimo)
    Bibliotecário --------- (Solicitar empréstimo)
    Bibliotecário --------- (Registrar devolução)
    Bibliotecário --------- (Cadastrar livro)
    Administrador --------- (Gerenciar usuários)

  Atores ficam fora da fronteira do sistema;
  casos de uso são representados por elipses dentro dela.
```

47. Crie um diagrama de classes com Aluno, Curso e Matricula. Defina atributos, operações e multiplicidades dos relacionamentos.

Um aluno pode possuir várias matrículas e um curso pode estar associado a várias matrículas. Cada matrícula pertence a exatamente um aluno e a exatamente um curso. A classe Matricula representa o relacionamento e pode guardar data e situação.

```
Aluno
- id: int
- nome: string
- email: string
+ consultarMatriculas()

Curso
- id: int
- nome: string
- cargaHoraria: int
+ consultarAlunos()

Matricula
- id: int
- data: date
- situacao: string
+ cancelar()
+ confirmar()

Multiplicidades:
Aluno 1 -------- 0..* Matricula
Curso 1 -------- 0..* Matricula

Cada Matricula referencia 1 Aluno e 1 Curso.
```

48. Modele um diagrama de sequência do processo de login, envolvendo usuário, interface, serviço de autenticação e banco de dados.

O usuário informa credenciais na interface. A interface envia os dados ao serviço de autenticação. O serviço consulta o usuário no banco e compara a senha informada com o hash armazenado, usando algoritmo apropriado. O resultado retorna à interface, que libera o acesso ou exibe uma mensagem genérica de falha.

```
Usuário       Interface       Serviço Auth.       Banco
   |               |                 |               |
   |-- credenciais->|                 |               |
   |               |-- autenticar -->|               |
   |               |                 |-- buscar user->|
   |               |                 |<- dados/hash --|
   |               |                 | valida senha   |
   |               |<- sucesso/erro -|               |
   |<- resultado ---|                 |               |
   |               |                 |               |

Em caso de sucesso, o serviço pode emitir uma sessão/token.
Nunca armazenar senha em texto puro.
```

49. Use uma ferramenta de modelagem de dados para desenhar um diagrama entidade-relacionamento de uma loja virtual com Cliente, Produto, Pedido e ItemPedido.

O modelo usa ItemPedido para resolver o relacionamento muitos-para-muitos entre Pedido e Produto. Um cliente pode realizar vários pedidos; cada pedido possui um ou mais itens; cada item referencia um produto. O preço_unitario em ItemPedido registra o preço praticado no momento da compra, mesmo que o preço atual do produto mude.

```
CLIENTE
- cliente_id (PK)
- nome
- email (UNIQUE)

PEDIDO
- pedido_id (PK)
- cliente_id (FK)
- data_pedido
- status

PRODUTO
- produto_id (PK)
- nome
- preco_atual
- estoque

ITEM_PEDIDO
- item_id (PK)
- pedido_id (FK)
- produto_id (FK)
- quantidade
- preco_unitario

Relacionamentos:
CLIENTE 1 -------- N PEDIDO
PEDIDO  1 -------- N ITEM_PEDIDO
PRODUTO 1 -------- N ITEM_PEDIDO
```

50. Proponha um sistema de agendamento de consultas com requisitos, arquitetura, linguagem, modelo de dados, diagramas CASE, testes e plano de quatro semanas.

Proposta: sistema web de agendamento para uma clínica. O paciente consulta horários disponíveis e solicita uma consulta; a recepção administra agenda e confirmações; profissionais consultam seus compromissos.

```
(a) Requisitos funcionais
RF01 — Cadastrar pacientes e seus dados de contato.
RF02 — Cadastrar profissionais, especialidades e horários de atendimento.
RF03 — Consultar horários disponíveis por profissional e data.
RF04 — Agendar, confirmar e cancelar consultas.
RF05 — Consultar a agenda diária e o histórico de consultas.

Requisitos não funcionais
RNF01 — Exigir autenticação e controlar acesso conforme o perfil.
RNF02 — Proteger dados pessoais e de saúde, com criptografia em trânsito e acesso restrito.
RNF03 — Responder às consultas de disponibilidade em até 2 segundos em condições normais de carga definidas.

(b) Arquitetura em três camadas
Apresentação: interface web para paciente, recepção e profissional.
Negócio: autenticação, regras de disponibilidade, prevenção de conflito de horários e agendamento.
Persistência: banco relacional para pacientes, profissionais, especialidades e consultas.

(c) Linguagem e justificativa
Python com Django no back-end e HTML/CSS/JavaScript no front-end. Django oferece estrutura para rotas, validação, autenti

(d) Modelo de dados
Paciente(paciente_id PK, nome, email, telefone)
Profissional(profissional_id PK, nome, especialidade_id FK)
Especialidade(especialidade_id PK, nome)
Consulta(consulta_id PK, paciente_id FK, profissional_id FK,
         inicio, fim, status)
Regra importante: impedir consultas sobrepostas para o mesmo profissional;
a validação deve ocorrer no serviço e ser protegida no banco/transação
sempre que possível.

(e) Dois diagramas CASE
1. Casos de uso: Paciente — consultar disponibilidade, agendar consulta,
   cancelar consulta; Recepcionista — cadastrar paciente, confirmar consulta;
   Profissional — consultar agenda.
2. Classes/entidades: Paciente, Profissional, Especialidade e Consulta,
   com as chaves e relacionamentos definidos no modelo de dados.

(f) Cinco testes
```

```
T01 — Agendamento em horário livre: deve criar consulta com status solicitado/confirmado.
T02 — Agendamento em horário ocupado: deve rejeitar conflito para o mesmo profissional.
T03 — Cancelamento de consulta existente: deve atualizar o status e liberar o horário conforme regra.
T04 — Consulta de agenda por profissional e data: deve retornar apenas compromissos correspondentes.
T05 — Acesso sem autenticação à agenda restrita: deve negar acesso.

Plano de desenvolvimento — quatro semanas
Semana 1: levantar requisitos, definir protótipos, arquitetura e modelo de dados.
Semana 2: implementar cadastro de pacientes/profissionais e autenticação.
Semana 3: implementar disponibilidade, agendamento, cancelamento e agenda.
Semana 4: executar testes, corrigir defeitos, revisar segurança, documentar e implantar versão inicial.
```

Observação: os diagramas foram apresentados em formato textual para facilitar a leitura e podem ser convertidos em diagramas gráficos em uma ferramenta CASE. As respostas são propostas didáticas; detalhes podem variar conforme os requisitos reais de cada projeto.